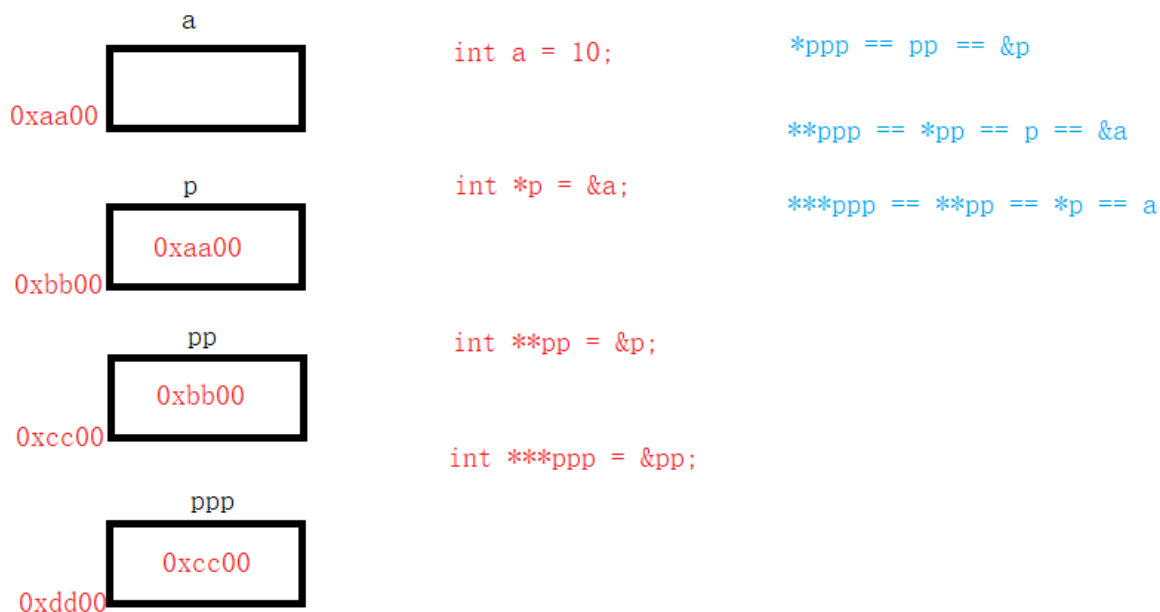


多级指针

- 多级指针不能跳跃定义。有一级，才能定义二级；有二级才能定义三级；有三级，才能定义4级。。。

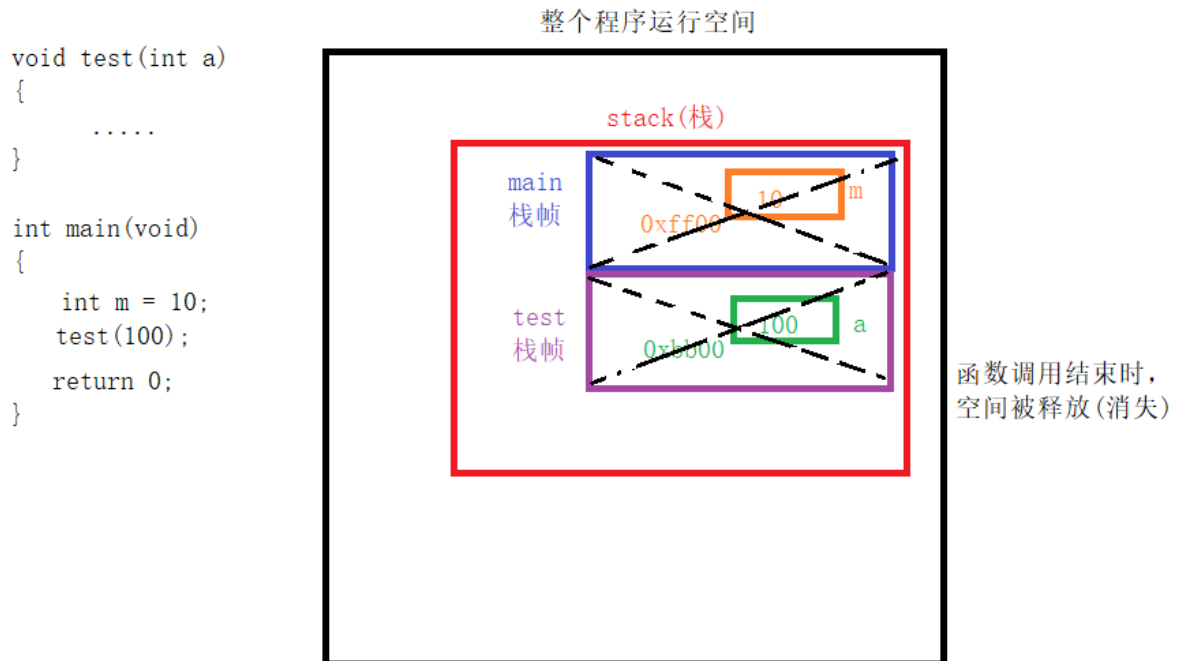
```
1  int a = 10;    // 普通变量
2  int *p = &a;    // 一级指针。是变量的地址。
3  int **pp = &p;   // 二级指针。是一级指针的地址。【重点】
4  int ***ppp = &pp; // 三级指针。是二级指针的地址。
5  int ****pppp = &ppp; // 四级指针。是三级指针的地址。
6  .....
7  *ppp == pp == &p;
8  **ppp == *pp == p == &a;
9  ***ppp == **pp == *p == a;
```



指针和函数

栈帧

- 当函数被调用时，系统会在 stack 空间上申请一块内存，用来给函数调用提供空间。存储形参和局部变量（定义在函数内部的变量）。
- 函数调用结束时，这块内存空间，会被自动释放（消失）。



传值和传址

传值(值传递)

- 函数调用期间，实参将自己的数据值，拷贝一份给形参。

```

// 定义函数，交换两个数。 -- 3杯水法
int swap1(int a, int b)    // a、b 形参。
{
    int tmp = 0; // 局部变量
    tmp = a;
    a = b;
    b = tmp;
}

```

```

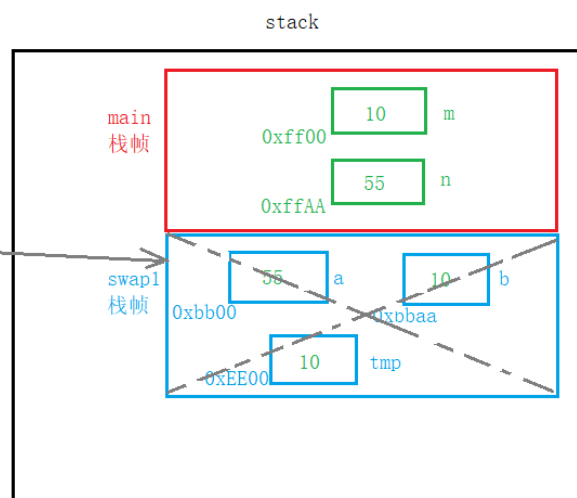
int main(void)
{
    int m = 10;
    int n = 55;

    // 调用 函数 swap1，交换两个数。

    swap1(m, n);    // m、n 实参。

    system("pause");
    return EXIT_SUCCESS;
}

```



传址(传引用)

- 函数调用期间，实参将自己的“地址值”，拷贝一份赋值给形参。
 - 可以在 A 栈帧中，借助地址，修改 B 栈帧上的变量数据。

```
// 定义函数，交换两个数。 -- 传引用（传址）
int swap2(int *a, int *b)    // a、b 形参。
{
    int tmp = 0;

    tmp = *a;
    *a = *b;
    *b = tmp;
}

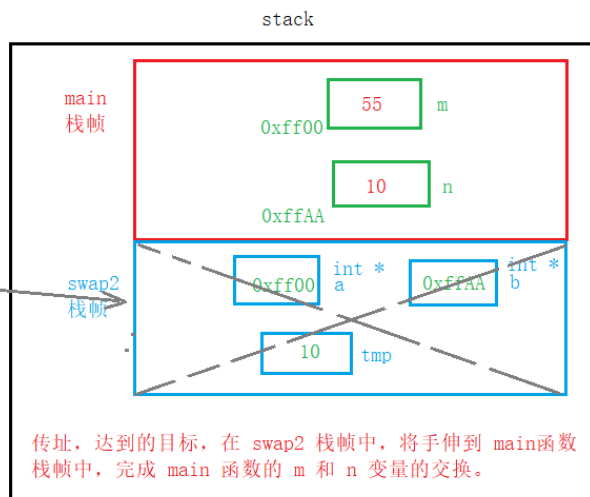
int main(void)
{
    int m = 10;
    int n = 55;

    swap2(&m, &n);

    system("pause");
    return EXIT_SUCCESS;
}
```

函数调用结束，栈帧消失，

但是，栈帧存在期间，已经修改了。main中的 m、n



数组做函数参数

- 数组做函数参数时，传递的不再是整个数组，而是数组的首地址（指针）。

```
1
2 int main(void)
3 {
4     int arr[] = {1, 4, 6, 7, 9, 0};
5
6     printf("main : sizeof(arr) = %u\n", sizeof(arr)); // 整个数组的大小。
7
8     // 调用 test 函数，传参 数组。
9     test(arr); // 实参!
10
11     system("pause");
12     return EXIT_SUCCESS;
13 }
14
15 // 定义函数，用数组做参数
16 void test(int arr[]) // 形参
17 {
18     printf("test : sizeof(arr) = %u\n", sizeof(arr)); // 指针的大小。
19
20     printf("arr[0] = %d\n", arr[0]);
21 }
```

- 当整型数组做函数参数时，通常在函数定义中，封装2个参数，一个表数组首地址，另一个表元素个数。

```
1 // 冒泡排序
2 // void BubbleSort(int arr[]) 这种传参，无法在函数内，求元素个数。
3 void BubbleSort(int arr[], int n)
4 {
5     for (int i = 0; i < n - 1; i++)
6     {
7         for (int j = 0; j < n - 1 - i; j++)
8         {
9             if (arr[j] > arr[j + 1])
```

```

10         {
11             int temp = arr[j];
12             arr[j] = arr[j + 1];
13             arr[j + 1] = temp;
14         }
15     }
16 }
17 }
18
19 int main(void)
20 {
21     int arr[] = {1, 4, 8, 12, 19, 43, 2, 7, 15};
22
23     // 获取元素个数
24     int n = sizeof(arr) / sizeof(arr[0]);
25
26     // 排序
27     BubbleSort(arr, n);
28
29     // 打印排序后结果
30     for (size_t i = 0; i < n; i++)
31     {
32         printf("%d ", arr[i]);
33     }
34     putchar('\n');
35
36     return 0;
37 }

```

- 数组做函数参数，也可以写成指针的形式。（本质一样）

```

1 void BubbleSort(int arr[], int n) == void BubbleSort(int *arr, int n)

```

指针做函数返回值

- 指针做函数返回值，不能返回【局部变量的地址】。
 - 当函数调用结束，栈帧空间释放，局部变量的地址，无效。
- 数组做函数返回值，不允许！（C语言中，只能写成 指针形式）

指针和字符串

基本知识

```

1 char str1[] = {'h', 'i', '\0'}; 【麻烦】 变量，可读可写。
2 char str2[] = "hi"; 变量，可读可写。
3 char *str3 = "hi"; 常量，只读。
4 char *str4 = {'h', 'i', '\0'}; // 错误！

```

演示 demo1:

- `char *str2 = "hello"` 是一个常量。不允许修改。

```
1 int main0401(void)
2 {
3     char str1[] = "hello"; // 相当于 {'h','e','l','l','o','\0'}
4     char* str2 = "";
5
6     str1[0] = 'R'; // 相当于 {'h','e','l','l','o','\0'}是变量，可以随意修改。
7     printf("%s\n", str1);
8
9     str2[0] = 'R'; // "hello" 是字符串常量。 不能修改!
10    printf("%s\n", str2);
11
12    system("pause");
13    return EXIT_SUCCESS;
14 }
```

演示 demo2:

- 同一个字符串常量，可以给多个不同的指针赋值。
- `char* str2 = "hello";` 和 `char* n = "hello";` 地址值都相同。都是字符串“hello”的地址。

```
1 int main(void)
2 {
3     char str1[] = "hello"; // 相当于 {'h','e','l','l','o','\0'}
4     char m[] = "hello";
5
6     char* str2 = "hello";
7     char* n = "hello";
8
9     printf("str1 = %p\n", str1);
10    printf("m    = %p\n", m); // 数组定义的 hello 地址不同。
11
12    printf("str2 = %p\n", str2);
13    printf("n    = %p\n", n); // 指针定义的 hello 是字符串常量，是同一个地址。
14
15    system("pause");
16    return EXIT_SUCCESS;
17 }
```

【结论】：当字符串(含有 `\0` 字符数组)，做函数参数时，不需要提供 2 个参数。因为每个字符串都有 `\0`。

练习

字符串比较 (strcmp()函数)

- 比较 `str1` 和 `str2`，如果相同返回0，不同则依次比较ASCII码，`str1 > str2` 返回1，否则返回 -1
 - 按对应的 位置，比较字符的大小。不比较ASCII 码 和。
- 分析：
 - 循环，依次比较两个 字符串 中 对应位字符。`\0` 结束。都相同 ---> 0

- 对应位不同，比较字符的 ASCII 码。str1 > str2 ----> 1, str1 < str2 ----> -1

```
1 // 数组的实现方式
2 int myStrcmp(char *str1, char *str2)
3 {
4     int i = 0;
5     while (str1[i] == str2[i]) // *(str1+i) == *(str2+i)
6     {
7         if (str1[i] == '\0')
8         {
9             return 0; // 2个字符串，一样！
10        }
11        i++;
12    }
13    // str1 和 str2 有字符不同。
14    return str1[i] > str2[i] ? 1 : -1;
15 }
```

```
1 // 指针的实现方式
2 int myStrcmp2(char* str1, char* str2)
3 {
4     while (*str1 == *str2) // *(str1+i) == *(str2+i)
5     {
6         if (*str1 == '\0')
7         {
8             return 0; // 2个字符串，一样！
9         }
10        str1++;
11        str2++;
12    }
13    // str1 和 str2 有字符不同。
14    return *str1 > *str2 ? 1 : -1;
15 }
```

字符串拷贝(strcpy函数)

- 将一个字符串中的所有字符，依次拷贝存放到另一个空字符串数组中。

```
1 // 数组版：
2 void myStrcpy(char* src, char* dst)
3 {
4     int i = 0;
5     while (src[i] != '\0') // while(src[i] != 0) while(strc[i])
6     {
7         dst[i] = src[i];
8         i++;
9     }
10    dst[i] = '\0'; //main中的 dst 初始化为0， 此步可以省略。
11 }
```

```

1 // 指针版
2 void myStrcpy2(char* src, char* dst)
3 {
4     while (*src != '\0') // while(src[i] != 0) while(strc[i])
5     {
6         *dst = *src;
7         src++;
8         dst++;
9     }
10    *dst = '\0'; //main中的 dst 初始化为0, 此步可以省略。
11 }

```

在字符串中查找字符出现的位置(strchr函数)

- “helloworld” ‘e’ ——> “elloworld”
 ‘l’ ——> “lloworld”

‘r’ ——> “rld”

```

1 // 指针版
2 char* myStrchr(char* str, char ch)
3 {
4     while (*str) //while (*str != '\0') == while (*str != 0)
5     {
6         if (*str == ch)
7         {
8             return str;
9         }
10        str++;
11    }
12    return NULL; // 在 str 中, 没有找到 ch
13 }

```

```

1 // 数组版
2 char* myStrchr2(char* str, char ch)
3 {
4     int i = 0;
5     while (str[i]) //while (str[i] != '\0') == while (str[i] != 0)
6     {
7         if (str[i] == ch) // str[i] == *(str+i)
8         {
9             return &str[i];
10        }
11        i++;
12    }
13    return NULL; // 在 str 中, 没有找到 ch
14 }

```

字符串去空格

- “ni chou sha ? chou ni za di !” ----> “nichousha?chounizadi!”

```

2 // 封装函数，去除字符串空格 --- 数组版
3 void str_no_space(char* src, char* dst)
4 {
5     int i = 0;        // 遍历 src 字符串
6     int j = 0;        // 记录 dst存储位置。
7     while (src[i])
8     {
9         if (src[i] != ' ') // 只有不为空格，才存储到 dst中。
10        {
11            dst[j] = src[i];
12            j++;          // 不为空格，j后移。为空格，j不动。
13        }
14        i++;
15    }
16    dst[j] = '\0';
17 }
18
19 // 封装函数，去除字符串空格 --- 指针版
20 void str_no_space2(char* src, char* dst)
21 {
22     while (*src)
23     {
24         if (*src != ' ') // 只有不为空格，才存储到 dst中。
25         {
26             *dst = *src;
27             dst++;      // 不为空格，指针后移。为空格，指针不动
28         }
29         src++;
30     }
31     *dst = '\0';
32 }
33
34 int main(void)
35 {
36     char str[] = "ni chou sha ? chou ni za di ! zai chou yi ge shi shi";
37     char dst[1024] = {0};
38
39     // 调用函数 去除 str中的空格，保存到 dst 中
40     // str_no_space(str, dst);
41     str_no_space2(str, dst);
42
43     printf("dst = %s\n", dst);
44
45     system("pause");
46     return EXIT_SUCCESS;
47 }

```

带参数的 main 函数

- 无参版：

```
1 | int main(void) { return 0; }
```


- 有参版：

```
1 int main(int argc, char *argv[]) { return 0; }
2     参1: 表示给 main 函数传递的参数的总个数。
3     参2: 是一个数组。数组的每一个元素都是 字符串 (char *)
```

测试代码：

```
1 int main(int argc, char *argv[])
2 {
3     int i = 0;
4
5     for (i = 0; i < argc; i++)
6     {
7         printf("argv[%d] = %s\n", i, argv[i]);
8     }
9
10    system("pause");
11    return EXIT_SUCCESS;
12 }
```

The screenshot shows a Visual Studio IDE with two windows. The left window displays the source code for '09-带参数的main.c', which is identical to the test code provided. The right window shows the command prompt output of running 'test.exe aa bb cc dd ee'. The output lists the values of argv[0] through argv[5], with argv[0] being 'test.exe' and argv[1] through argv[5] being 'aa', 'bb', 'cc', 'dd', and 'ee' respectively. A red box highlights the argv array values in the terminal output.

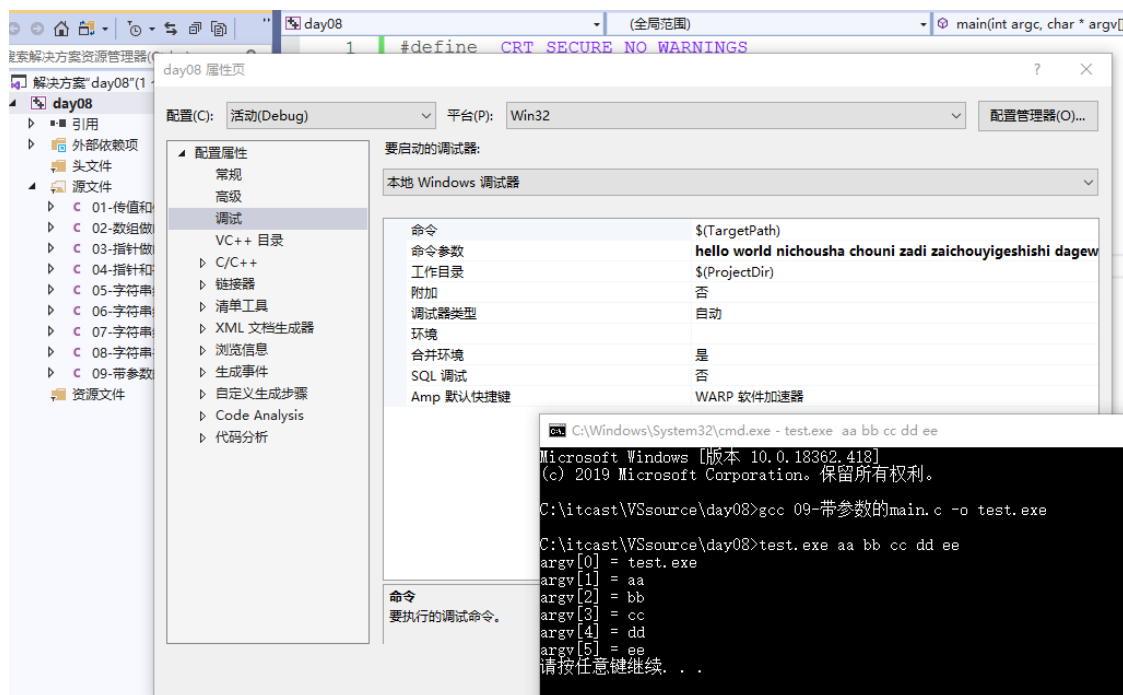
argv[i]	Value
argv[0]	"test.exe"
argv[1]	"aa"
argv[2]	"bb"
argv[3]	"cc"
argv[4]	"dd"
argv[5]	"ee"

测试方法

1. 在终端中，使用 gcc 编译得到可执行文件，如 test.exe
2. 不能获取带有空格的字符串!!! 空格是分隔符。

```
1 gcc 09-带参数的main.c -o test.exe
2
3 test.exe aa bb cc dd ee
4 test.exe hello world haha xixi hoho heihei .....
5 argc: --- 6
6 argv[0] = test.exe
7 argv[1] = aa
8 argv[2] = bb
9 argv[3] = cc
10 argv[4] = dd
11 argv[5] = ee
```

2. 在 VS 中，项目名称上 右键 —— 属性 —— 配置属性——调试 —— 命令行参数 —— 写入 待测试的命令行参数。



字符串练习

str 中 substr 出现的次数

str : "hellollollollollo "

substr: "llo"

写函数测试， llo 在 "hellollollollollo " 出现了多少次。

学习、使用 strstr() 函数。再实现上述的 题目。

求字符串非空格元素个数

"ni chou sha ? chou ni za di ! zai chou yi ge shi shi" 统计这里，除空格外 字符的个数。

字符串逆置

"hello" ---> "olleh"

"word" ----> "drow"

判断字符串是回文

abcba --- 是回文

amkilolikma --- 是回文。

abccba ---- 是回文。

abcdba --- 不是回文。